

Traffic Violation Detection on Two-Wheelers

A Computer Vision Pipeline using YOLO26 and OCR

Project Report

May 2026

Model Link: <https://huggingface.co/rajcreo/traffic-violation-models/tree/main>

Team Member

Ritu Raj Sharma (MT2025721)

Shwetank Singh (MT2025725)

Table of Contents

Table of Contents	2
1. Introduction	4
2. Dataset	4
2.1 Source and Composition	4
2.2 Class Distribution	4
2.3 Object Size Analysis	5
2.4 Scene Complexity	6
2.5 Sample Images	7
3. Model Architecture and Design	9
3.1 Single-Model Approach	9
3.2 Why YOLO26	9
3.3 Association Logic	9
3.4 OCR	9
4. Training	10
4.1 Setup	10
4.2 Augmentation	10
5. Training Results (YOLO26s)	10
5.1 Loss Curves	10
5.2 mAP Convergence	10
5.3 Precision and Recall	11
5.4 Per-Class Performance	11
5.5 Learning Rate and Training Dynamics	13
5.6 Gradient and Weight Analysis	13
5.7 Training Time	14
6. Pipeline Architecture	15
7. Test Set Evaluation	16
7.1 Detection Metrics on Test Set	16
8. End-to-End Pipeline Evaluation	17
8.1 Evaluation Setup	17
8.2 Results	17
8.3 Failure Analysis	17
9. OCR Analysis	18
10. Known Issues and Limitations	19
11. Conclusion	20

1. Introduction

This report describes the design, implementation, and evaluation of a computer vision pipeline for detecting traffic rule violations on two-wheelers (motorcycles and scooters) from street photographs. The system identifies vehicles with more than two riders or riders not wearing helmets, and reads their license plates.

A violation is defined as either having more than 2 riders on a single two-wheeler or any rider not wearing a helmet. For each violating vehicle, the pipeline reports the number of riders, the number without helmets, and the license plate text. Non-violating vehicles are excluded from the output.

The output follows a structured JSON format with a *violations* array, where each entry contains *num_riders*, *helmet_violations* (count of riders without helmets), and *license_plate* (OCR text). Images with no violations return an empty array.

The evaluation metric is a weighted combination of violation detection accuracy and OCR edit distance, with weights of 0.5 each (tentatively, may shift to 0.4/0.6 favoring OCR). Inference is expected to complete within approximately 5 seconds per image with no internet access.

2. Dataset

2.1 Source and Composition

The primary dataset is "Bike Helmet Detection" from Roboflow, containing 8,466 images annotated in YOLO format with four classes: bike (0), helmet (1), no-helmet (2), and number-plate (3). All four classes are annotated in the same label files, which is important since the pipeline uses a single-model approach rather than cascaded detection. After subsampling same-camera frames, the working dataset contains 6,521 images.

The dataset includes a mix of image types: real street views from top/side/back angles (~3,500 images), single bike front views (~500), person face closeups (~500), dashcam/GoPro footage (~700), bicycles with bicycle helmets (~300), street photos of walking pedestrians (~50), and a small number of night views and video call screenshots.

Same-camera street view frames were subsampled from ~3,500 to ~1,500 to reduce viewpoint overfitting. Face closeup images were kept because the pipeline checks for a bike bounding box first, so a face without a detected bike produces no output. Similarly, bicycle images only carry helmet/no-helmet labels (no bike class), so they contribute to helmet feature learning without generating false positive bike detections.

2.2 Class Distribution

The dataset was split into train (80%), validation (10%), and test (10%) sets using stratified sampling with source grouping to prevent data leakage from same-camera frames. The final split contains 5,216 training images (26,234 annotations), 652 validation images (3,409 annotations), and 653 test images (3,061 annotations).

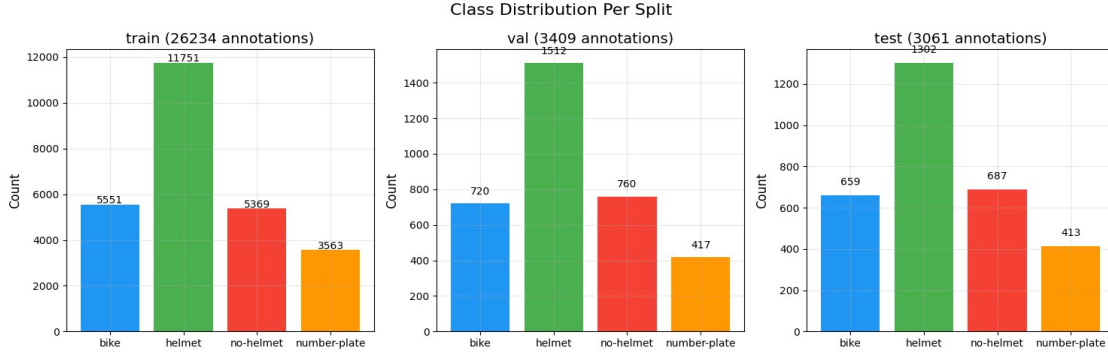


Figure 1: Annotation counts per class across train, validation, and test splits.

Class proportions are consistent across all three splits, confirming the stratified sampling worked correctly. Bikes account for about 21%, helmets 44%, no-helmet 21%, and number-plates 13% in each split. Helmets are the most frequent class because a single image often contains multiple helmeted riders.

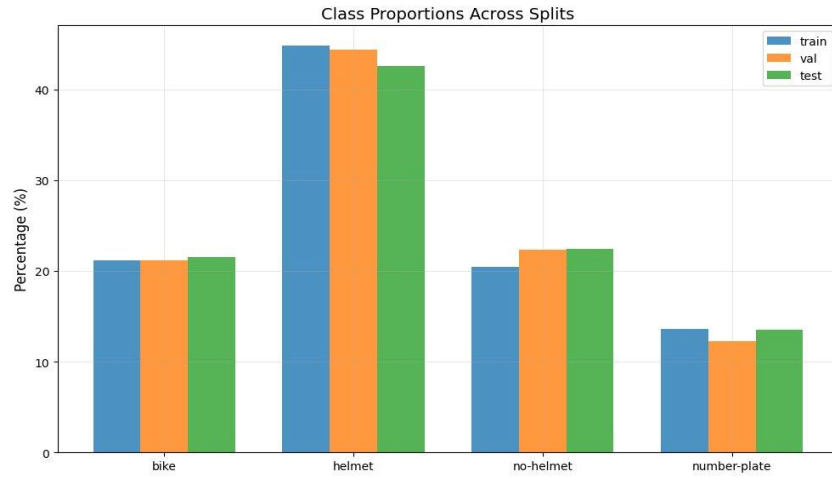


Figure 2: Class proportions (%) across splits. Distributions are well-balanced.

2.3 Object Size Analysis

Bounding box sizes vary substantially across classes. Bikes have a median area of ~120,000 pixels (at 1280px input resolution), while helmets, no-helmets, and number plates are much smaller with median areas of 3,696, 3,738, and 3,053 pixels respectively. This two-order-of-magnitude size difference is a key challenge and motivated the choice of `imgsz=1280` and a model architecture with small-target-aware label assignment.

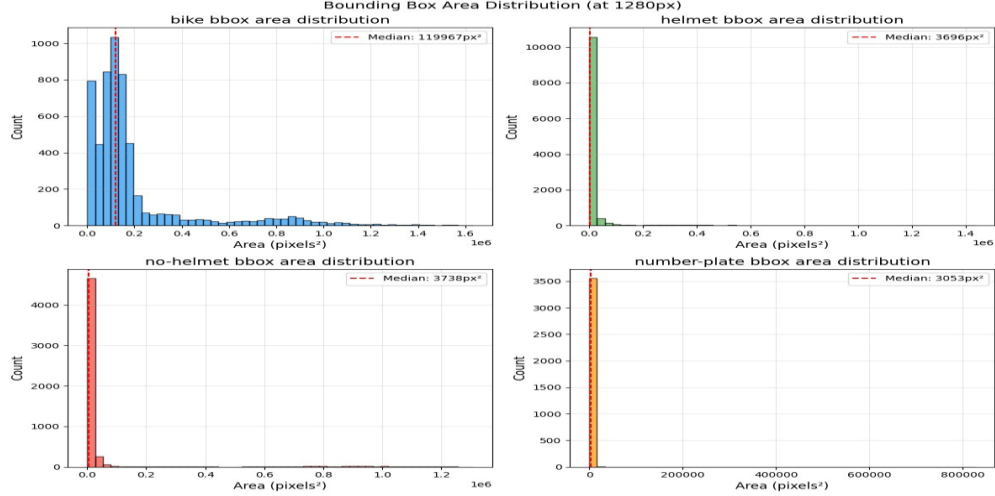


Figure 3: Bounding box area distributions per class at 1280px resolution.

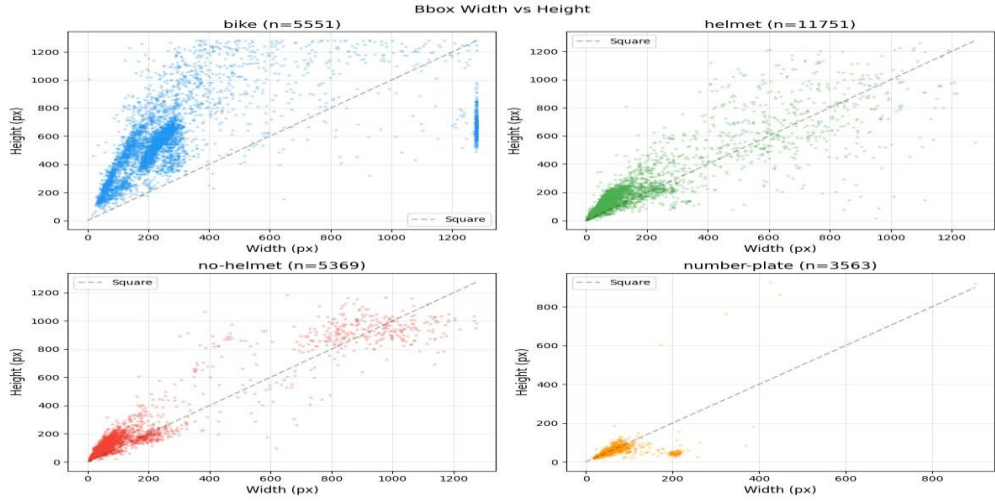


Figure 4: Width vs height scatter plots. Bikes tend toward tall rectangles; plates are wide and short.

2.4 Scene Complexity

Most images contain 1-5 annotations, with a long tail extending to 40+ objects in crowded street scenes. The co-occurrence matrix shows that helmet and bike annotations frequently appear together (2,034 co-occurrences), which is expected since helmeted riders sit on bikes. The violation profile of the training set shows that 28.9% of images contain violations, 16.2% have bikes but no violations, and 54.9% have no bike at all.

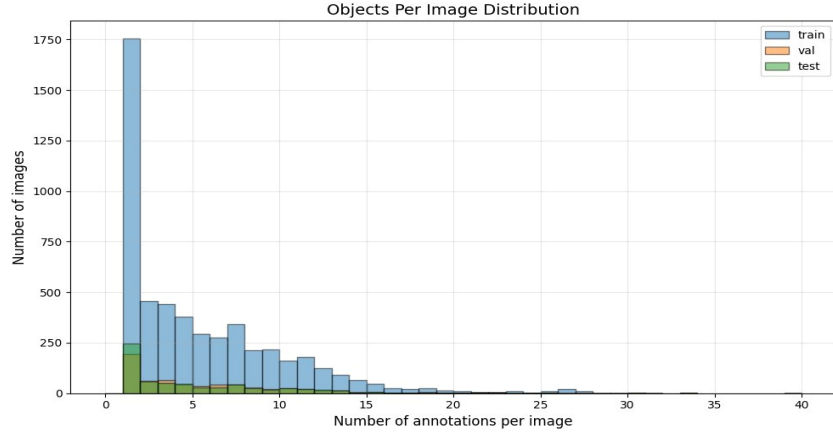


Figure 5: Distribution of annotation counts per image.

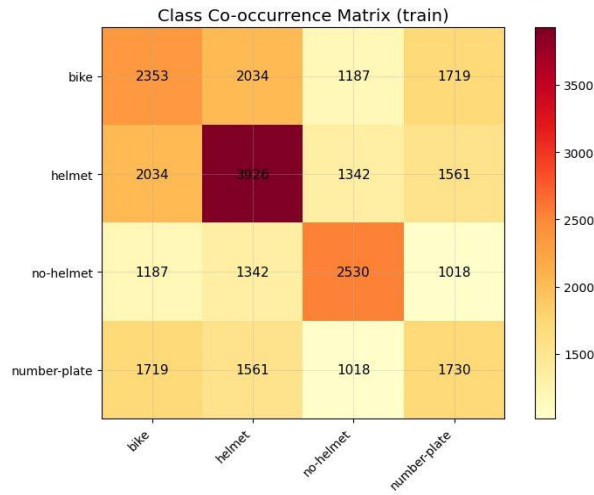


Figure 6: Class co-occurrence matrix for the training split.

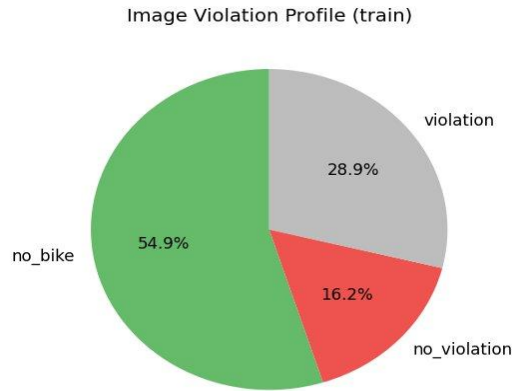


Figure 7: Proportion of images containing violations, non-violating bikes, and no bikes.

2.5 Sample Images

The figure below shows representative training images with ground truth annotations overlaid. The dataset covers a range of viewpoints including overhead street cameras, dashcam footage, front-facing shots, and

closeup portraits. Some images contain face closeups labeled only with no-helmet, which helps the model learn bare-head features without producing false positive bike detections.

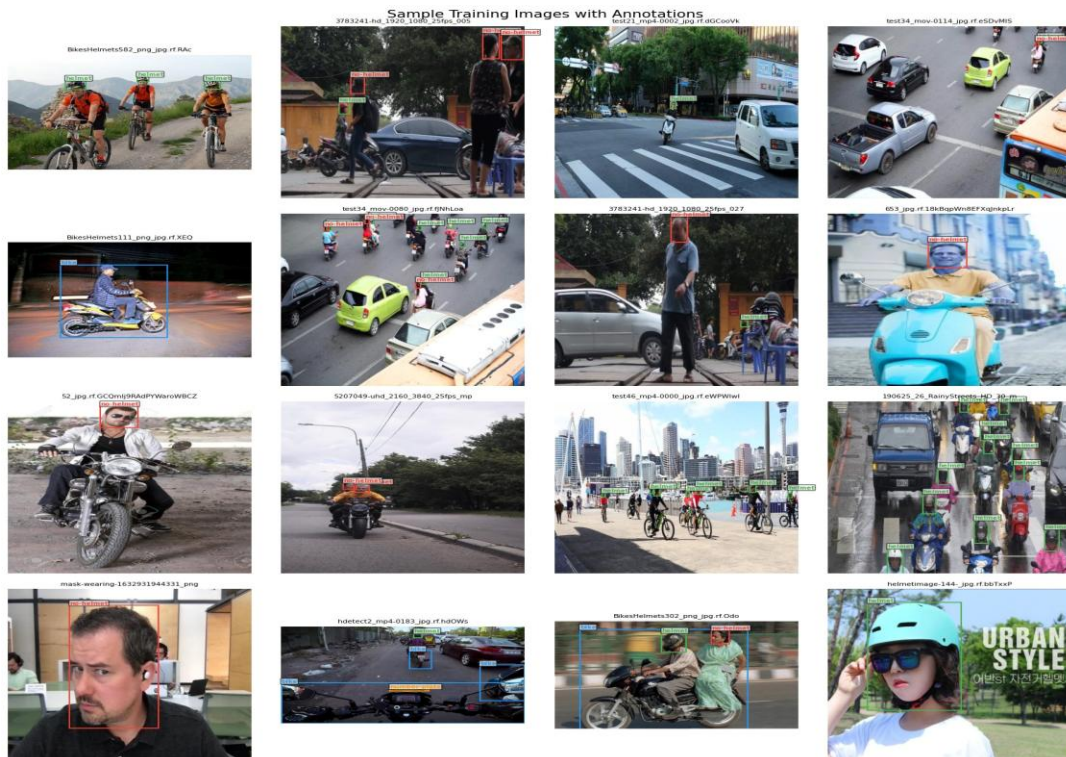


Figure 8: Sample training images with bounding box annotations.

3. Model Architecture and Design

3.1 Single-Model Approach

The pipeline uses a single YOLO26 model that detects all four classes (bike, helmet, no-helmet, number-plate) simultaneously, rather than a two-stage cascade where a first model detects bikes and a second model analyzes crops for helmets and plates.

The two-stage approach was considered but rejected for several reasons. First, a missed bike in stage 1 means zero detections downstream, creating a cascading failure. Second, crop quality is fragile since tight crops clip helmets at the edges while loose crops bleed in neighboring vehicles. Third, coordinate remapping between stages introduces errors. Finally, the dataset already has all four classes annotated in the same label files, making single-model training straightforward.

YOLO26 is well-suited for this because its Small-Target-Aware Label Assignment (STAL) ensures that small objects like helmets and plates receive proper training signal even when large bike bounding boxes dominate the image. Running at `imgsz=1280` provides 4x the pixel area compared to the standard 640, further helping with small object detection.

3.2 Why YOLO26

YOLO26 was selected over YOLOv11 and YOLOv12 based on several factors. Compared to YOLOv11, YOLO26 offers STAL for small objects and 43% faster CPU inference, with NMS-free operation and stable training. YOLOv12 was rejected due to training instability and fragmented repository support.

3.3 Association Logic

After detection, the pipeline associates' helmet, no-helmet, and plate detections with their parent bike using spatial reasoning. For each bike bounding box, the system finds helmet/no-helmet detections whose center falls inside that bike box, with geometric priors requiring that head-related detections fall within the upper 70% of the bike box and plates within the lower 60%.

Rider count is computed as the sum of helmet and no-helmet detections per bike (there is no separate "rider" or "person" class in the dataset). When bikes overlap, detections are assigned to the nearest bike center. Rider count is capped at 5 as a sanity check. Plates are assigned using a combination of overlap area and confidence scoring.

3.4 OCR

License plate text is extracted using a dual-engine approach. PaddleOCR serves as the primary engine due to its speed (~50-100ms per crop) and strong performance on clean plates. EasyOCR runs as a fallback only when PaddleOCR confidence falls below 0.3. If both engines produce low-confidence results, the system retries with slight rotation adjustments. The pipeline always returns whatever text is available, since even low-confidence OCR output scores better on edit distance than an empty string.

4. Training

4.1 Setup

Training was conducted on Kaggle GPU. Both YOLO26s and YOLO26l were trained for 50 epochs with patience=10 for early stopping, using mixed precision (AMP). The optimizer was set to "auto", which selected AdamW with $lr=0.00125$ and momentum=0.9. Note that the auto optimizer ignores manually specified learning rate values.

YOLO26l was trained at batch size 8 with $imgsz=1280$. YOLO26s used batch size 16 at the same image size. Full parameter fine-tuning was used.

4.2 Augmentation

Data augmentation included HSV jittering ($h=0.015$, $s=0.7$, $v=0.4$), horizontal flipping (50%), mosaic augmentation (100%), and moderate scaling (0.3). Vertical flipping, rotation, and shear were disabled since bikes do not appear upside-down or heavily rotated in real street scenes.

5. Training Results (YOLO26s)

5.1 Loss Curves

The training and validation loss curves show steady convergence over 50 epochs. Box loss and classification loss both decrease consistently, with training loss continuing to drop while validation loss plateaus after epoch ~30, indicating the model is approaching its capacity on this data. The DFL loss column is present but negligible (values ~0.003-0.01), consistent with YOLO26 predicting box coordinates directly rather than using distribution focal loss.

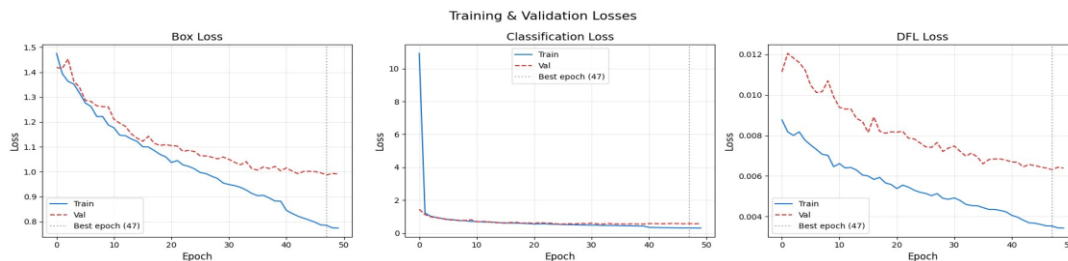


Figure 9: Training and validation losses (box, classification, DFL) over 50 epochs.

5.2 mAP Convergence

$mAP@50$ reached 0.9449 at the best epoch (47), with $mAP@50-95$ reaching 0.711. Both metrics show rapid improvement in the first 10 epochs, then gradual gains through epoch 35, after which they plateau. Early stopping did not trigger within the 10-epoch patience window.

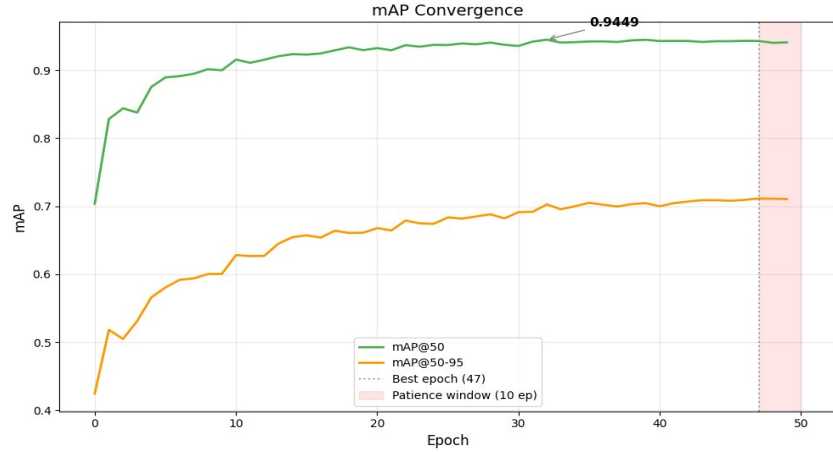


Figure 10: $mAP@50$ and $mAP@50-95$ convergence. Best epoch marked at 47.

5.3 Precision and Recall

Precision and recall both climb steadily from ~ 0.68 to ~ 0.90 over training. Precision edges slightly ahead of recall in later epochs, reaching 0.897 vs 0.888 at the best epoch. This slight precision advantage is acceptable for this task since false positives (reporting a non-violation as a violation) are generally worse than false negatives.

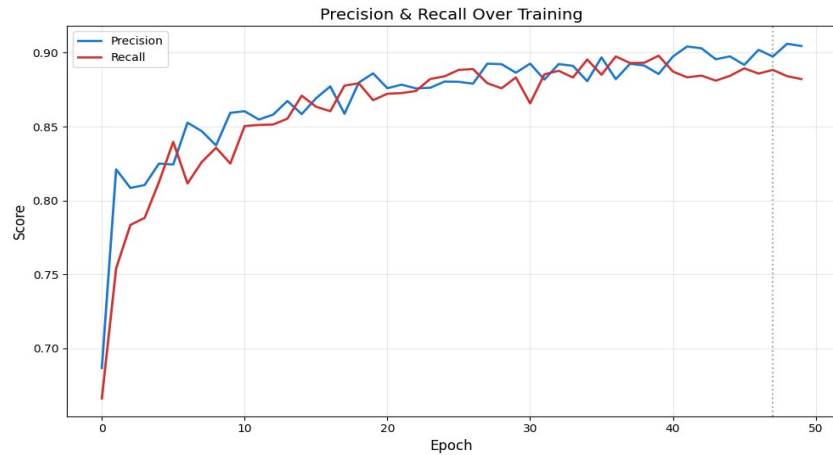


Figure 11: Precision and recall progression over training.

5.4 Per-Class Performance

Per-class AP@50 on the validation set at the best epoch (47) shows that number-plate detection is strongest at 0.964, followed by helmet at 0.955, no-helmet at 0.930, and bike at 0.923. The bike class has the lowest AP, which is the primary bottleneck in the pipeline since a missed bike means the entire vehicle is missed.

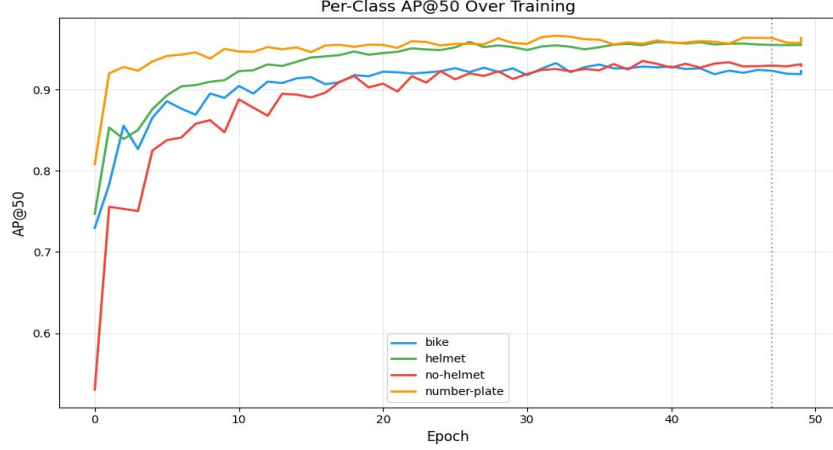


Figure 12: Per-class AP@50 progression on validation set. Bike (blue) consistently lags behind other classes.

Table 1: Per-class validation AP@50 at best epoch for both models.

Model	Bike	Helmet	No-helmet	Plate
YOLO26s	0.9233	0.9552	0.9297	0.9636
YOLO26l	0.9239	0.9549	0.9400	0.9590

Model	mAP@50	mAP@50-95	Precision	Recall
YOLO26s	0.9429	0.7111	0.8974	0.8878
YOLO26l	0.9444	0.7207	0.9043	0.8948

Table 2: Overall validation metrics comparison. Differences between s and l are marginal.

The difference between YOLO26s and YOLO26l is minimal across all metrics. The large model gains less than 0.2% on mAP@50 and about 1% on mAP@50-95. For the bike class specifically, the gap is only 0.06% (0.9233 vs 0.9239). The most notable difference is in no-helmet AP where the large model is about 1% better. Given the marginal gains and the practical advantages of a smaller model (faster inference, lower memory), YOLO26s was selected as the primary model for fine-tuning and deployment.

5.5 Learning Rate and Training Dynamics

The auto optimizer selected AdamW with an initial learning rate of 0.00125, which decays linearly over training. All three parameter groups (pg0/biases, pg1/conv weights, pg2/batch norm) follow the same schedule after the warmup phase. The first 3 epochs use a warmup where biases start at 0.1 and decrease while other parameters ramp up from near-zero.

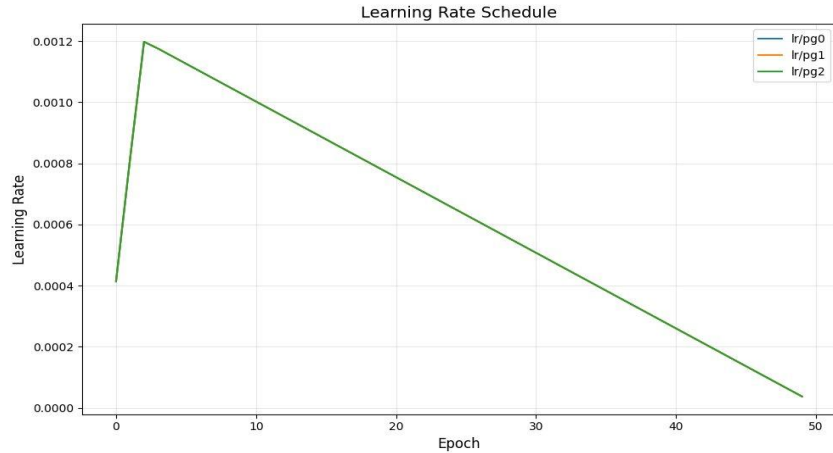


Figure 13: Learning rate schedule over training.

5.6 Gradient and Weight Analysis

The gradient L2 norms reveal a concerning pattern in the detection head. While backbone and neck gradients remain stable throughout training (L2 norms around 500-1500), the head gradient norm explodes from ~3,000 in the early epochs to ~17,000 by epoch 50. This suggests the detection head is overfitting or encountering numerical instability in the final training phases. The mean absolute gradient plot confirms this trend with head gradients rising sharply after epoch 40.

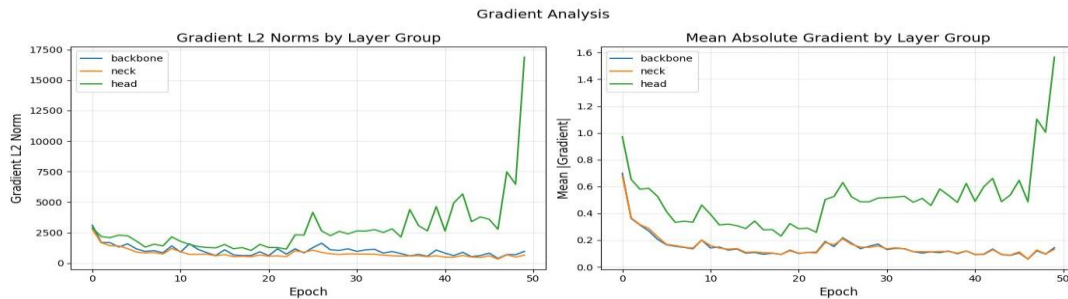


Figure 14: Gradient norms by layer group. Head gradients explode in the final 10 epochs.

Weight norms show a more stable picture. The backbone drifts the most from its pretrained initialization (~18% change), followed by the neck (~12%) and head (~4.5%). The head changing least makes sense since it was already randomly initialized for this task. Weight changes plateau after epoch 30-35, suggesting the model has largely converged by that point.

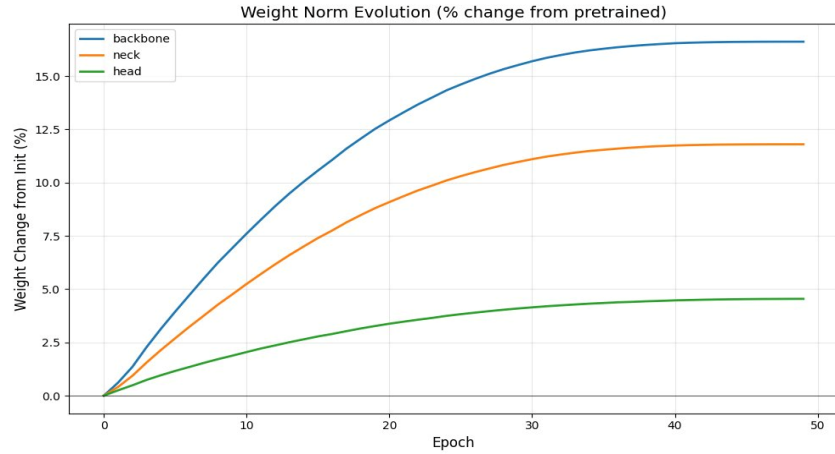


Figure 15: Weight norm evolution as percentage change from pretrained initialization.

5.7 Training Time

Each epoch took approximately 106 seconds (mean 106.4s), making the full 50-epoch training run about 89 minutes. The first epoch was slightly longer at 127 seconds due to initial data loading and caching.

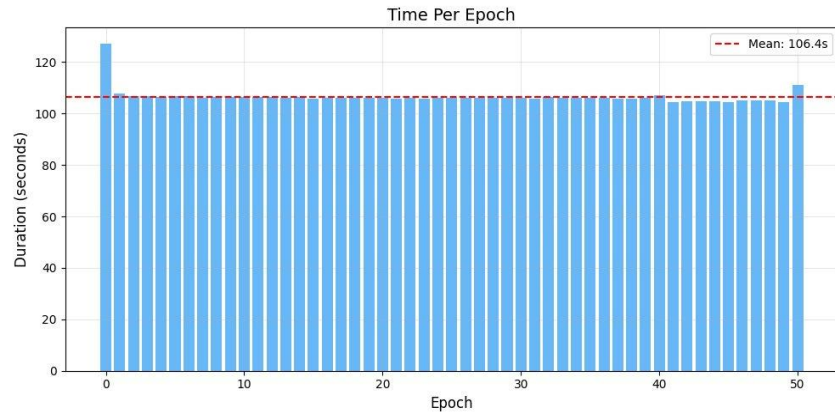


Figure 16: Time per epoch. Consistent ~106s after initial warmup.

6. Pipeline Architecture

The complete inference pipeline follows this sequence: the input image is passed through YOLO26 at `imgsz=1280` (GPU) or `640` (CPU), which outputs bounding boxes for all four classes. For each detected bike, the system finds associated helmet, no-helmet, and plate detections using the spatial rules described in Section 3.3. Rider count is computed as helmet + no-helmet detections per bike. A violation is flagged if rider count exceeds 2 or if any no-helmet detections exist.

For violating bikes only, the associated plate region is cropped, preprocessed, and passed through the dual OCR engine. The final output is a JSON object with one entry per violating vehicle. The entire pipeline is implemented in a single `solution.py` file with a `TrafficViolationDetector` class exposing a `predict(image_path)` method. All model paths are relative to a `model_dir` parameter, and the `predict` method never raises exceptions, returning an empty violations list on any failure.

The system adapts to available hardware: it uses GPU with `imgsz=1280` when CUDA is available and falls back to CPU with `imgsz=640` otherwise. Inference speed is approximately 185ms per image on GPU (excluding OCR), well within the 5-second guideline.

7. Test Set Evaluation

7.1 Detection Metrics on Test Set

The final YOLO26s model was evaluated on the held-out test set of 653 images (3,061 annotations). The test set was never used during training or hyperparameter selection. Results are shown below.

Table 3: Test set detection results.

Class	Precision	Recall	mAP@50	mAP@50-95
bike	0.856	0.877	0.890	0.720
helmet	0.912	0.891	0.896	0.664
no-helmet	0.898	0.886	0.879	0.651
number-plate	0.928	0.911	0.925	0.682
All	0.899	0.891	0.897	0.679

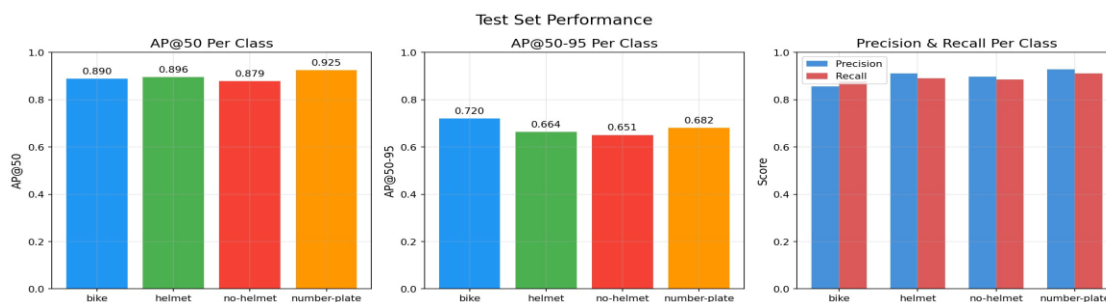


Figure 17: Per-class AP@50, AP@50-95, and precision/recall on the test set.

Test set mAP@50 is 0.897, approximately 4-5% lower than validation mAP@50 (0.943). This gap is expected since the validation set was used for model selection (best epoch checkpoint), while the test set provides an unbiased estimate of generalization. The relative ranking of classes is consistent: number-plate has the highest AP@50 (0.925), followed by helmet (0.896), bike (0.890), and no-helmet (0.879). Bike detection remains the weakest class, confirming the pattern observed during training.

Precision and recall are well balanced across all classes, with no class showing a large gap between the two. Number-plate detection has the highest precision (0.928), while helmet detection has the highest recall differential favoring precision (0.912 vs 0.891). The overall inference speed is 21ms preprocessing, 185ms inference, and 0.6ms postprocessing per image on GPU.

8. End-to-End Pipeline Evaluation

8.1 Evaluation Setup

The complete pipeline was evaluated end-to-end on 43 manually annotated images drawn from multiple sources, including the test split and the NCKH dataset. Ground truth was created by visually inspecting each image and recording the number of riders, helmet violations, and license plate text for every violating two-wheeler. This evaluation tests the full chain: detection, spatial association, violation logic, and OCR.

8.2 Results

Table 4: End-to-end pipeline evaluation metrics.

Metric	Score
Violation detection rate	0.586
Rider count score	0.414
Helmet count score	0.517
OCR exact match	0.310
OCR edit distance (normalized)	0.636
Final combined score ($w_1=0.5$, $w_2=0.5$)	0.551

8.3 Failure Analysis

Out of 43 evaluated images, the pipeline produced 17 failure cases across different categories. Missed violations (10 cases) were the most common failure mode, where the pipeline failed to detect either the bike or a no-helmet rider, causing the entire violation to be unreported. This aligns with bike detection being the weakest class at 0.890 AP@50.

Wrong rider counts (5 cases) and wrong helmet counts (2 cases) indicate errors in the spatial association logic, where detections were either missed due to the geometric priors (upper 70% / lower 60% constraints) or incorrectly assigned to neighboring bikes in crowded scenes. False positives (3 cases) occurred when the model detected a violation where none existed, typically from misclassifying a helmet as no-helmet.

OCR failures (7 cases) were caused by low-resolution plate crops, angled or partially occluded plates, and non-standard plate formats. The OCR exact match rate of 31% is low, but the normalized edit distance of 0.636 shows that even when the OCR output is not perfectly correct, it captures a significant portion of the plate text. This is important because the evaluation scoring uses edit distance rather than exact match.

9. OCR Analysis

License plate recognition uses PaddleOCR v5 as the primary engine with EasyOCR as a fallback. Plate crops are preprocessed before OCR: resized to a standard height of 64 pixels maintaining aspect ratio, converted to grayscale, enhanced with CLAHE for contrast normalization, and smoothed with a mild Gaussian blur to reduce noise.

PaddleOCR processes the crop first. If its confidence exceeds 0.3, the result is accepted. Otherwise, EasyOCR runs on the same crop. If both engines produce low-confidence results, the system retries PaddleOCR with small rotation adjustments at $\pm 5^\circ$ and $\pm 10^\circ$. The best result across all attempts is returned, since even a partially correct plate reading scores better on edit distance than an empty string.

The OCR exact match rate of 31% and edit distance score of 0.636 reflect the inherent difficulty of reading license plates from street images. Common failure modes include low resolution (plates far from camera), motion blur, angled views where characters are foreshortened, partial occlusion by other vehicles or objects, and non-standard fonts on Indian plates. The dual-engine approach with rotation retry helps recover some marginal cases, but fundamentally the OCR accuracy is bounded by the quality of the plate crop produced by the detection stage.

10. Known Issues and Limitations

Gradient explosion in the detection head during the last 10 epochs of YOLO26l training suggests possible overfitting. Head gradient L2 norms rose from ~3,000 to ~17,000 between epochs 40 and 50, while backbone and neck gradients remained stable. This did not noticeably hurt final mAP since the best checkpoint was saved at epoch 47 before the worst spikes, but it indicates the model could benefit from fewer epochs or gradient clipping.

Missed violations account for the largest share of pipeline errors (10 out of 17 failures). When the model fails to detect a bike, the entire vehicle and all its riders are invisible to the downstream pipeline. This cascading failure is inherent to the single-model architecture and is the primary area for improvement.

The rider counting approach (helmet + no-helmet = riders) has an inherent limitation: a rider whose head is fully occluded by another rider will not be counted. This is a dataset limitation since there are no separate rider or person annotations. In the pipeline evaluation, wrong rider counts accounted for 5 of 17 failures.

OCR performance is the weaker component of the pipeline. The 31% exact match rate means that nearly 7 out of 10 plates are read with at least one character error. This is partly a detection issue (low-resolution crops) and partly an OCR limitation (PaddleOCR and EasyOCR are general-purpose engines, not trained specifically on Indian license plates). A plate-specific OCR model trained on Indian plate formats could improve this significantly.

The fine-tuning process on the NCKH dataset revealed that class mapping errors can cause catastrophic forgetting in a single epoch, and that the Ultralytics auto optimizer ignores user-specified learning rates. These were resolved by verifying label mappings before training and explicitly setting the optimizer to AdamW.

11. Conclusion

This report presented a single-model traffic violation detection pipeline that detects bikes, helmets, bare heads, and license plates simultaneously using YOLO26s, then applies spatial association logic and dual-engine OCR to produce structured violation reports.

The detection model achieves mAP@50 of 0.897 on the held-out test set, with per-class AP@50 ranging from 0.879 (no-helmet) to 0.925 (number-plate). The end-to-end pipeline achieves a violation detection rate of 58.6%, a normalized OCR edit distance of 0.636, and a combined score of 0.551. Missed violations and OCR errors are the primary failure modes.

The model was initially trained on 5,216 images from the Roboflow Bike Helmet Detection dataset and subsequently fine-tuned with 761 additional motorcycle images from the NCKH 2023 dataset to improve bike detection diversity. YOLO26s was selected over YOLO26l due to nearly identical accuracy (0.15% mAP difference) with significantly lower inference cost.

The main areas for improvement are bike detection recall (to reduce missed violations) and OCR accuracy (to improve plate reading). A specialized Indian license plate OCR model and a larger, more diverse training dataset for bike detection would likely yield the most significant gains.